# CMSC 25750: Large Language Models
# Homework 2: LLM Alignment

Instructor: Chenhao Tan

Due: Apr 17, 2026, 11:59 pm

This homework explores three core alignment techniques applied to the same summarization task: **Supervised Fine-Tuning (SFT)**, **Instruction Tuning**, and **Reinforcement Learning from Human Feedback (RLHF)** via GRPO. By running all three methods on the same model and dataset, you will observe how each technique shapes model behavior differently and understand the trade-offs involved.

**Getting Started** We host all the code snippets and relevant files on this GitHub repo. Pull the assignment repository to get all necessary files:

```
git pull git@github.com:uchicago-llm-course/spring2026-assignments.git
```

**Submission** Export your `hw2.ipynb` to a PDF file, and then upload to Gradescope with all cell outputs visible. This PDF is a read-only problem sheet; do **not** submit it.

**Repository layout**

```
HW2/
|-- hw2.ipynb              (submit this; run all cells, write answers here)
|-- hw2.pdf                (this problem sheet, read only)
|-- compile.py             (run to train models and generate cache/ files)
|-- src/
|   |-- sft.py             (implement Section 1)
|   |-- instruction_tuning.py  (implement Section 2)
|   |-- rlhf.py            (implement Section 3)
|-- tests/
|   |-- test_sft.py        (autograder; do not modify)
|   |-- test_instruction_tuning.py  (autograder; do not modify)
|   |-- test_rlhf.py       (autograder; do not modify)
|-- cache/                 (auto-generated by compile.py)
|-- pyproject.toml
```

**General Instructions** See `README.md` for the environment setup. You can work one section at a time: implement, test, compile, then fill in the notebook.

1. Implement the coding problems in `src/sft.py`, `src/instruction_tuning.py`, and `src/rlhf.py`.

2. Run the autograder for the completed section:

```
pytest tests/test_sft.py -v                    # after Section 1
pytest tests/test_instruction_tuning.py -v     # after Section 2
pytest tests/test_rlhf.py -v                   # after Section 3
```

3. Compile to train models and generate cache files (requires GPU; see `README.md` for SLURM instructions):

```
python compile.py section1   # or section2 / section3 / all
```

4. Run the corresponding cells in `hw2.ipynb` and write your answers in the designated markdown cells.

5. When all sections are done, export `hw2.ipynb` as PDF and submit to Gradescope.

**Shared Setup** All three sections use the same configuration:

- **Model**: `gpt2` (117M parameters, GPT-2 base)
- **Dataset**: CNN/DailyMail (abisee/cnn_dailymail, 3.0.0) — 5,000 training examples, 500 validation examples
- **Task**: News summarization (article → summary)
- **Evaluation**: ROUGE-1 F1 score

**Acknowledgements**
This assignment is partially adapted from Understanding Language Models (Franke, Eickhoff, Tsvilodub) and CS336 Assignment 5 (Stanford). GRPO is described in Shao et al. (2024).

## Section 1: Supervised Fine-Tuning (SFT) (30 pts)

Fine-tuning a pre-trained language model on a labelled dataset is the foundation of the modern alignment pipeline. In **supervised fine-tuning (SFT)**, we continue training with standard maximum-likelihood on (input, output) pairs and update all model weights.

In this section you will implement the simplest version: the training sequence concatenates the article and the summary with a separator, and the cross-entropy loss is applied to *all* token positions. Sections 2–3 will improve upon this baseline.

**Training sequence format.**

$$\texttt{article} + \texttt{SEPARATOR} + \texttt{summary} + \texttt{[EOS]}$$

where `SEPARATOR = " TL;DR: "`.

**Truncation.** If the combined token count exceeds `max_length = 256`, trim the article from the right so the summary is always preserved in full. Pad shorter sequences on the right to exactly `max_length` tokens.

### Coding Problems

**1.1: Implement `tokenize_for_sft` (6 pts)**

In `src/sft.py`, implement `tokenize_for_sft(article, summary, tokenizer, max_length)`.
Concatenate the article, `SEPARATOR`, summary, and EOS token. Apply right-padding to `max_length`. The **labels** tensor must equal **input_ids** at every non-padded position (loss on all tokens), and be $-100$ at padding positions.
Return a `dict` with keys `"input_ids"`, `"attention_mask"`, and `"labels"`, each a `LongTensor` of shape `(max_length,)`.

**1.2: Implement `compute_sft_loss` (6 pts)**

In `src/sft.py`, implement `compute_sft_loss(model, input_ids, attention_mask, labels)`.
Run a forward pass through the HuggingFace CausalLM with the provided `labels` tensor. The model automatically ignores $-100$ positions. Return the scalar loss.

**ROUGE.** **ROUGE** (Recall-Oriented Understudy for Gisting Evaluation) is an automatic metric for text summarization that measures $n$-gram overlap between a generated text and one or more human references. **ROUGE-1** counts unigram (single-word) matches:

$$\text{ROUGE-1 F1} = \frac{2 \cdot P \cdot R}{P + R}, \quad P = \frac{|\text{pred} \cap \text{ref}|}{|\text{pred}|}, \quad R = \frac{|\text{pred} \cap \text{ref}|}{|\text{ref}|}$$

where $|\cdot|$ denotes the number of unigrams. Scores range from 0 (no overlap) to 1 (perfect overlap). We use the HuggingFace `evaluate` library to compute it:

```
import evaluate
rouge = evaluate.load("rouge")
result = rouge.compute(predictions=["the cat sat"], references=["the cat sat"])
# result["rouge1"] == 1.0
```

**1.3: Implement `evaluate_rouge`** **(6 pts)**

In `src/sft.py`, implement `evaluate_rouge(model, tokenizer, articles, ground_truth_summaries, device, max_new_tokens, num_examples)`.
For each of the first `num_examples` articles:

1. Build a prompt: `article + SEPARATOR` (no summary).
2. Tokenize and move to `device`.
3. Generate up to `max_new_tokens` tokens with greedy decoding (`do_sample=False`).
4. Decode only the newly generated tokens (not the prompt) to obtain the predicted summary.

Compute ROUGE-1 F1 using the `evaluate` library. Return a dict with `"rouge1"` (mean F1), `"predictions"`, and `"references"`.

### 1.4 Analysis

Run `python compile.py section1` (requires GPU), then open `hw2.ipynb` and execute the Section 1 cells. The notebook loads the cached results and displays training curves and ROUGE scores automatically.

(a). **[4 pts]** Inspect the training loss curve. Does the loss decrease across epochs? What does a decreasing loss tell us about what the model is learning?

**Deliverable**: 2–3 sentences.

(b). **[4 pts]** A simple non-trained baseline is the *first-sentence* baseline: return the first sentence of the article as the "summary". Compare your SFT ROUGE-1 score against this baseline. What does the comparison tell you about the difficulty of the task?

**Deliverable**: 2–3 sentences.

(c). **[4 pts]** In the current implementation, the loss is computed on *all* tokens (article + separator + summary). Why might computing loss on the article tokens be suboptimal for an alignment objective? What signal are those tokens providing to the optimizer?

**Deliverable**: 2–3 sentences.

## Section 2: Instruction Tuning (35 pts)

**Instruction tuning** extends SFT in two concrete ways:

1. **Instruction template.** Inputs are wrapped in a natural-language instruction, teaching the model to follow instructions rather than merely continue text.
2. **Response masking.** The cross-entropy loss is computed *only* on the response (summary) tokens. All instruction and article tokens receive a label of $-100$ and are excluded from the gradient. This focuses learning entirely on generating high-quality outputs.

Instruction tuning is the standard first step in modern alignment pipelines (InstructGPT, Llama-2-Chat, Llama-3-Instruct). Models fine-tuned this way exhibit much better zero-shot instruction following than plain SFT models.

**Instruction template.**

```
Summarize the following article:

{article}

Summary: {summary}
```

The prompt fed to the model during generation is everything up to and including `"Summary:"` (no summary text). During training the full string is used, but only the summary portion contributes to the loss.

### Coding Problems

**2.1: Implement `format_instruction`** **(5 pts)**

In `src/instruction_tuning.py`, implement `format_instruction(article, summary=None)`.
When `summary` is `None`, return:

```
INSTRUCTION_TEMPLATE.format(article=article)
```

When `summary` is provided, append `" " + summary` to the template string.
This function is used both to build training sequences (with summary) and generation prompts (without summary).

**2.2: Implement `tokenize_for_instruction_tuning`** **(8 pts)**

In `src/instruction_tuning.py`, implement `tokenize_for_instruction_tuning(article, summary, tokenizer, max_length)`.
**Key requirement — response masking**: the **labels** tensor must be $-100$ for all tokens belonging to the instruction template and the article (everything up to and including `"Summary:"`), and equal to **input_ids** for all summary and EOS tokens.
Implementation outline:

1. Tokenize the prompt-only string to find the boundary index.

2. Tokenize the full string (prompt + `" "` + summary + EOS).
3. Truncate to `max_length` if needed, trimming the article section (not the summary).
4. Pad to `max_length` on the right.
5. Build labels: $-100$ for prompt tokens and padding, **input_ids** for summary tokens.

### 2.3 Analysis

(a). **[5 pts]** Compare the ROUGE-1 scores of the SFT model and the IT model. Also inspect the qualitative examples shown in the notebook. In what qualitative ways do the summaries differ? Which model produces more instruction-following outputs?

**Deliverable**: 3–4 sentences with numbers.

(b). **[5 pts]** Suppose you remove response masking and compute the loss on *all* tokens (as in Section 1), but keep the instruction template. Would you expect the model to learn instruction following? Why or why not?

**Deliverable**: 2–3 sentences.

### 2.4 Written Questions

(a). **[5 pts]** Modern instruction-tuned models like Llama-3.1-Instruct use a structured *chat template* (e.g., `<|begin_of_text|><|start_header_id|>user<|end_header_id|>...`). Why do models need a standardized chat template? What problems could arise if users prompt an instruction-tuned model without its expected template?

**Deliverable**: 2–3 sentences.

(b). **[7 pts]** Instruction tuning typically uses a curated set of high-quality (instruction, response) pairs (e.g., 52K examples in Alpaca, 15K in LIMA). Suppose you instead collected 1M random (instruction, response) pairs scraped from the internet with no quality filtering. Would training on more data necessarily improve instruction following?

**Deliverable**: 2–3 sentences.

## Section 3: RLHF via GRPO (35 pts)

After instruction tuning, models can be further improved by **reinforcement learning from human feedback (RLHF)**: rather than maximizing the likelihood of a fixed training set, the model is trained to maximize a *reward signal* that reflects human preferences.

**The standard RLHF pipeline.** The classic RLHF pipeline used in InstructGPT consists of three stages:

1. **SFT**: fine-tune the base model on high-quality demonstrations (Section 1).
2. **Reward model training**: collect human preference comparisons between model outputs and train a separate *reward model* $r_\phi$ to predict which output a human would prefer.
3. **RL fine-tuning**: use Proximal Policy Optimization (PPO) to update the policy $\pi_\theta$ to maximize the reward model's score while staying close to the SFT policy via a KL-divergence penalty.

Stage 2 is expensive: it requires collecting thousands of human preference annotations and training an additional large model. In this homework we replace it with an automatic proxy reward — ROUGE-1 F1 — so that the full pipeline can run without human annotators.

**Policy gradient and PPO.** The key idea of policy gradient methods is to reinforce completions that received high reward and suppress those that received low reward. Naively computing $\nabla_\theta \mathbb{E}[r(o)]$ leads to high-variance estimates; PPO stabilizes training with two tricks:

- **Importance sampling.** Rather than sampling new completions at every gradient step, PPO reuses completions from an *old* policy $\pi_{\theta_{\text{old}}}$ and corrects for the distribution shift with the ratio $r_t(\theta) = \pi_\theta(o_t|x)/\pi_{\theta_{\text{old}}}(o_t|x)$.
- **Clipping.** To prevent the policy from changing too drastically, the ratio is clamped to $[1-\varepsilon,\, 1+\varepsilon]$ before multiplying by the advantage, giving the surrogate objective $\mathcal{L}_{\text{clip}}$.

PPO also requires training a *value network* (critic) to estimate expected future reward and compute advantages $A_t = r_t - V(s_t)$. This adds significant complexity and memory cost.

**GRPO: eliminating the critic.** GRPO (Shao et al., 2024) removes the value network entirely. Instead of estimating $V(s_t)$ with a learned critic, GRPO estimates the *baseline* empirically: for each prompt $x$ it samples $G$ completions, and normalizes each reward relative to the mean and variance of that group. This group mean acts as a Monte-Carlo baseline that is unbiased under the assumption that all $G$ completions come from the same prompt distribution. GRPO was used in DeepSeekMath within DeepSeek-R1.

Formally, for each prompt $x$, $G$ completions $\{o_1, \ldots, o_G\}$ are sampled from the current policy. Their rewards $\{r_1, \ldots, r_G\}$ are group-normalized:

$$A_i = \frac{r_i - \text{mean}_j(r_j)}{\text{std}_j(r_j) + \varepsilon}, \qquad \varepsilon = 10^{-8}$$

The full GRPO objective combines the PPO-clip surrogate with a KL divergence penalty against a fixed *reference policy* $\pi_{\text{ref}}$ (the instruction-tuned checkpoint before RL training begins):

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\mathbb{E}_{x,\{o_i\}}\left[\min\left(\frac{\pi_\theta(o_i|x)}{\pi_{\theta_{\text{old}}}(o_i|x)}\,A_i,\;\; \text{clip}\left(\frac{\pi_\theta(o_i|x)}{\pi_{\theta_{\text{old}}}(o_i|x)},\, 1-\varepsilon,\, 1+\varepsilon\right) A_i\right)\right] + \beta\, \mathbb{D}_{\text{KL}}\left[\pi_\theta \,\middle\|\, \pi_{\text{ref}}\right]$$

where $\pi_{\theta_{\text{old}}}$ is the policy frozen at the start of each update step and $\beta$ is a scalar KL coefficient (default $\beta = 0.04$ in `trl`). The KL penalty anchors the policy to $\pi_{\text{ref}}$, limiting how much it can

change in any direction. In `trl`'s `GRPOTrainer` this penalty is applied automatically; the `beta` field of `GRPOConfig` controls its strength.

**How the building blocks connect.** The three functions you implement in Problems 3.1–3.3 correspond directly to the three core steps of the GRPO update:

1. `compute_rouge_reward` (Problem 3.1) evaluates each completion and returns a scalar reward $r_i \in [0, 1]$.
2. `compute_group_normalized_rewards` (Problem 3.2) takes all $G \times$ (batch size) rewards and converts them to advantages $A_i$ by normalizing within each group of $G$.
3. `compute_grpo_clip_loss` (Problem 3.3) takes the per-token log-probabilities under the current and old policies and the advantages, and returns the clip-term loss $\mathcal{L}_{\text{clip}}$ to be minimized.

In Problem 3.4 you will play with `run_grpo_training` using `trl`'s `GRPOTrainer`, which adds the KL penalty, manages the training loop, gradient accumulation, and checkpoint saving.

**Reward function.** We use ROUGE-1 F1 as a proxy reward: the model is rewarded for producing summaries that overlap with the human-written reference. Unlike a learned reward model, ROUGE requires no annotation — any generated summary can be scored automatically. This makes it an attractive stand-in for experimentation, though Section 3.5 will ask you to critically evaluate its limitations.

## Coding Problems

**3.1: Implement `compute_rouge_reward`** **(5 pts)**

In `src/rlhf.py`, implement `compute_rouge_reward(completions, ground_truths)`.
Use the `evaluate` library to compute ROUGE-1 F1 for each (completion, ground_truth) pair independently. Return a `list` of `float` values in $[0, 1]$.

**3.2: Implement `compute_group_normalized_rewards`** **(8 pts)**

In `src/rlhf.py`, implement `compute_group_normalized_rewards(rewards, group_size)`.
Reshape the flat list of $N$ rewards into shape $(N/G,\ G)$, compute per-group mean and standard deviation, and normalise. Return a `FloatTensor` of shape $(N, )$. Use $\varepsilon = 10^{-8}$ to avoid division by zero.

**3.3: Implement `compute_grpo_clip_loss`** **(7 pts)**

In `src/rlhf.py`, implement `compute_grpo_clip_loss(log_probs, old_log_probs, advantages, clip_epsilon=0.2)`.
Compute the importance-sampling ratio $r = \exp(\log \pi_\theta - \log \pi_{\text{old}})$, clip it to $[1-\varepsilon,\ 1+\varepsilon]$, and return the negative mean of $\min(r \cdot A,\ r_{\text{clip}} \cdot A)$ as a scalar loss tensor.

**3.4: Implement `run_grpo_training`** **(5 pts)**

In `src/rlhf.py`, implement `run_grpo_training(ckpt_path, train_dataset, tokenizer, output_dir)`.
Using `trl`'s `GRPOTrainer`, wire together your three building blocks (`build_grpo_reward_fn`, `GRPOConfig`, `GRPOTrainer`) to run GRPO fine-tuning starting from the instruction-tuned checkpoint.
Configure the trainer with:

- `learning_rate = 1e-5`
- `max_steps = 200`
- `per_device_train_batch_size = 4`
- `num_generations = 4` ($G = 4$ completions per prompt)
- `max_new_tokens = 60`

The function should call `trainer.train()` and return the trainer's `state.log_history` (a list of dicts logged at each step). The notebook will visualize the returned logs — no plotting code goes in this function.

### 3.5 Analysis

(a). **[4 pts]** Report the final ROUGE-1 F1 scores for all three methods (SFT, Instruction Tuning, RLHF) in a table. Which method achieves the highest ROUGE-1 score? Does this match your expectation? Provide one possible explanation for the ordering.

**Deliverable**: a table plus 2–3 sentences.

(b). **[3 pts]** Look at the GRPO reward curve plotted in the notebook. Does the mean reward increase during training? What would a flat or decreasing reward curve indicate?

**Deliverable**: 2–3 sentences.

(c). **[3 pts]** ROUGE measures $n$-gram overlap with a reference summary. Describe two failure modes where a model could achieve a high ROUGE score while producing a *poor* summary. This is an example of *reward hacking* — the model optimising the proxy metric rather than the true objective.

**Deliverable**: 2 concrete failure modes, one sentence each.